Praktikum Pemrorgraman

Pirantik Bergerak



Muhammad Yusuf

2409106093

C 2024

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2025

Deskripsi Singkat Aplikasi

JerseyStore adalah aplikasi mobile berbasis Flutter yang digunakan untuk menampilkan dan membeli berbagai produk jersey. Aplikasi menyediakan fitur **Home, Explore, pencarian produk, keranjang belanja, pengaturan jumlah produk, checkout, dan Profile**. Aplikasi juga menerapkan konsep **Stateful Widget** untuk mengelola perubahan data seperti pencarian, navigasi halaman, jumlah produk, dan proses checkout.



```dart
class AppShell extends StatefulWidget {
  const AppShell({super.key});

  @override
  State<AppShell> createState() => _AppShellState();
}

class _AppShellState extends State<AppShell> {
  // State: indeks tab yang sedang dipilih (0 Home, 1 Explore, 2 Cart, 3 Profile).
  int currentIndex = 0;

  // setState() memberi tahu Flutter bahwa state berubah -> build() diulang.
  void selectPage(int index) {
    setState(() {
      currentIndex = index;
    });
  }
```

AppShell menggunakan StatefulWidget karena memiliki data yang dapat berubah, yaitu currentIndex. Nilai tersebut digunakan untuk menentukan halaman yang sedang aktif. Ketika pengguna memilih menu pada BottomNavigationBar, setState() digunakan untuk mengubah currentIndex sehingga tampilan diperbarui.

```dart
class ExplorePage extends StatefulWidget {
  const ExplorePage({super.key});

  @override
  State<ExplorePage> createState() => _ExplorePageState();
}

class _ExplorePageState extends State<ExplorePage> {
  // Controller TextField pencarian (untuk tombol clear).
  late final TextEditingController searchController;

  // State: kata kunci pencarian (huruf kecil).
  String searchQuery = '';

  // initState: siapkan controller sekali saat halaman dibuat.
  @override
  void initState() {
    super.initState();
    searchController = TextEditingController();
  }
```

ExplorePage menggunakan StatefulWidget karena memiliki state berupa searchQuery yang berubah ketika pengguna melakukan pencarian. initState() digunakan untuk membuat TextEditingController satu kali ketika halaman dibuat.

```dart
TextField(
  controller: searchController,
  onChanged: (value) {
    setState(() => searchQuery = value.toLowerCase());
  },
```

TextField digunakan untuk menerima input pencarian. Setiap kali pengguna mengetik, onChanged dijalankan dan setState() mengubah nilai searchQuery. Perubahan state tersebut membuat tampilan produk diperbarui sesuai kata pencarian.

```dart
class ProductExploreCard extends StatefulWidget {
  const ProductExploreCard({super.key, required this.product});

  final Product product;

  @override
  State<ProductExploreCard> createState() => _ProductExploreCardState();
}

class _ProductExploreCardState extends State<ProductExploreCard> {
  // Controller untuk membaca/menulis isi TextField jumlah.
  late final TextEditingController quantityController;

  // State lokal: jumlah yang akan ditambahkan ke keranjang.
  int quantity = 1;

  // initState: dijalankan SEKALI saat State dibuat -> siapkan controller.
  @override
  void initState() {
    super.initState();
    quantityController = TextEditingController(text: '$quantity');
  }
```

ProductExploreCard menggunakan StatefulWidget karena jumlah produk (quantity) dapat berubah berdasarkan input pengguna. State quantity menyimpan jumlah produk yang ingin dimasukkan ke keranjang.

```dart
void updateQuantity(String value) {
  final parsed = int.tryParse(value);
  if (parsed == null || parsed < 1) return;

  final clamped = parsed.clamp(1, widget.product.stock).toInt();
  setState(() {
    quantity = clamped;
  });
```

Fungsi updateQuantity() digunakan untuk mengubah jumlah produk berdasarkan input pengguna. setState() digunakan setelah nilai quantity berubah agar Flutter membangun ulang tampilan dengan jumlah terbaru.

```dart
class CartProductCard extends StatefulWidget {
  const CartProductCard({
    super.key,
    required this.product,
    required this.quantity,
    required this.maxQuantity,
    required this.onQuantityChanged,
    required this.onRemove,
  });

  final Product product;
  final int quantity;
  final int maxQuantity; // jumlah di keranjang + sisa stok
  final ValueChanged<int> onQuantityChanged;
  final VoidCallback onRemove;

  @override
  State<CartProductCard> createState() => _CartProductCardState();
}

class _CartProductCardState extends State<CartProductCard> {
  late final TextEditingController quantityController;

  // initState: isi awal TextField sesuai jumlah di keranjang.
  @override
  void initState() {
    super.initState();
    quantityController = TextEditingController(text: '${widget.quantity}');
  }
```

CartProductCard menggunakan StatefulWidget karena memiliki state lokal berupa TextEditingController. initState() digunakan untuk mengatur nilai awal input jumlah produk berdasarkan quantity yang diterima dari halaman keranjang.

```dart
  @override
  void didUpdateWidget(covariant CartProductCard oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.quantity != widget.quantity &&
        quantityController.text != '${widget.quantity}') {
      quantityController.text = '${widget.quantity}';
    }
  }
```

didUpdateWidget() digunakan ketika data yang diterima oleh CartProductCard berubah dari luar widget. Fungsi ini memastikan isi TextField tetap sesuai dengan jumlah produk terbaru.

```dart
int addToCart(Product product, {int quantity = 1}) {
  final added = quantity.clamp(0, product.stock).toInt();
  if (added == 0) return 0;

  product.stock -= added;
  cartQuantities[product.id] = quantityOf(product) + added;
  notifyListeners(); // beri tahu semua widget yang memantau state ini
  return added;
}
```

Selain state lokal pada StatefulWidget, aplikasi juga menggunakan ChangeNotifier untuk menyimpan state keranjang secara terpusat. Ketika data keranjang berubah, notifyListeners() memberi tahu widget yang menggunakan state tersebut agar tampilan diperbarui.

link video dokumentasi : https://drive.google.com/drive/folders/1keiKiOYnabUmEcT-YumAPDWxcsCTrgug?usp=sharing